UniAgent: An End-to-End Decentralized Framework for Autonomous Agent Discovery, Delegation, and Payment

Ryoto Taga
ryoto.taga12@gmail.com
Independent Researcher
Tokyo, Japan

M. Fahim Ferdous Khan
khan@iniad.org
Faculty of Information Networking for Innovation and Design (INIAD)
Toyo University
Tokyo, Japan

Abstract

Large language model (LLM) agents are increasingly able to act on behalf of users, but the infrastructure that lets them find, hire, and pay one another across organizational boundaries is still fragmented. Open protocols already exist for the individual pieces: the Agent2Agent (A2A) protocol standardizes agent-to-agent communication, x402 turns the HTTP 402 status code into a micropayment channel, and ERC-8004 records agent identity on chain. However, these protocols have so far been demonstrated only in isolation or as one-to-one proofs of concept, and no end-to-end service architecture ties them together so that a single natural-language request is autonomously decomposed into agent discovery, delegation, and payment. In this paper we present *UniAgent*, a decentralized agent marketplace that integrates A2A, x402, and ERC-8004 under an LLM-based orchestration layer. A three-layer design separates a *marketplace layer* (an on-chain ERC-8004 registry kept in sync with an off-chain cache through blockchain event webhooks), an *orchestration layer* (a ReAct agent that discovers, inspects, and invokes external agents through three tools), and a *payment layer* (x402 with delegated EIP-3009 signatures). We describe a working prototype deployed on the Base Sepolia testnet and report an end-to-end evaluation over 20 flight- and hotel-search tasks. UniAgent shows that open communication, on-chain identity, and protocol-native micropayments can be combined into a coherent, vendor-neutral marketplace in which user tasks are carried out across separately deployed A2A services.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Information systems** → *Web services*; • **Software and its engineering** → *Distributed systems organizing principles.*

Keywords

AI agents, agent marketplace, A2A protocol, x402, ERC-8004, micropayments, on-chain identity, LLM orchestration, ReAct

ICCCM '26, Tokyo, Japan

ACM ISBN 979-8-4007-2439-8
https://doi.org/XXXXXXX.XXXXXXX

ACM Reference Format:
Ryoto Taga and M. Fahim Ferdous Khan. 2026. UniAgent: An End-to-End Decentralized Framework for Autonomous Agent Discovery, Delegation, and Payment. In *Proceedings of The 14th International Conference on Computer and Communications Management (ICCCM 2026) (ICCCM '26).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large language models (LLMs) have made it practical to build autonomous agents that plan, call tools, and complete multi-step tasks on behalf of a user. As these agents proliferate, a natural next step is for one agent to delegate work to another: a travel assistant might hire a specialized flight agent and a separate hotel agent, each operated by a different organization. Realizing this vision requires three capabilities that cut across trust boundaries: agents must be able to *communicate* with one another, to be *discovered* and *identified*, and to *pay* for services without a shared account or a manual contract.

Open standards have recently emerged for each of these capabilities. The Agent2Agent (A2A) protocol [10] defines how agents describe themselves through an *Agent Card* and exchange messages over JSON-RPC. The x402 protocol [8] reuses the long-dormant HTTP 402 "Payment Required" status code to attach stablecoin micropayments to ordinary web requests. ERC-8004 [9] provides an on-chain registry of agent identities, building on the ERC-721 token standard so that any party can look up an agent without trusting a central directory. Individually, these protocols are compelling; together, they suggest a decentralized alternative to the closed agent stores offered by today's platforms.

Despite this progress, three gaps remain. **First**, A2A standardizes communication but explicitly leaves *discovery*—how a client finds an appropriate agent in the first place—outside its scope. **Second**, recent work by Vaziry et al. [11] demonstrated the technical feasibility of combining A2A, x402, and on-chain Agent Cards, but did so at the protocol level, as one-to-one interactions between a fixed pair of agents; it did not provide a service architecture that takes a user's high-level task and autonomously drives discovery, delegation, and payment to completion. **Third**, centralized platforms such as the OpenAI GPT Store [6] do offer end-to-end experiences, but they confine the agent ecosystem to plug-ins inside a single vendor's walled garden [3], with platform-specific registration, discovery, and billing.

The result is a missing middle: the individual protocols exist, and closed platforms show what a smooth user experience looks like,

but no *open* architecture connects the protocols through an orchestration layer that can turn a natural-language request into cross-organizational agent transactions. This motivates our research question:

> *How can open agent communication, on-chain identity, and protocol-level micropayments be integrated into a cohesive marketplace architecture with LLM-based orchestration, so that a user's high-level task is autonomously decomposed into agent discovery, delegation, and payment across organizational boundaries?*

We answer this question with *UniAgent*, a decentralized agent marketplace whose architecture is summarized in Figure 1. UniAgent takes the registration, discovery, and payment functions that a centralized platform bundles together and re-implements them on open protocols—A2A and x402—and a standardized on-chain identity layer—ERC-8004. On top of these, an LLM ReAct agent [13] acts as an orchestration layer that analyzes the task, discovers candidate agents, inspects their specifications, selects among them, and executes them with automatic payment.

Contributions. This paper makes three contributions:

(1) **An integrated marketplace architecture** that unifies A2A, x402, and ERC-8004 behind an orchestration layer, organized as three loosely coupled layers (marketplace, orchestration, payment).
(2) **An autonomous LLM pipeline** in which a ReAct agent drives task analysis, agent discovery, specification inspection, and paid execution through three tools (`discover_agents`, `fetch_agent_spec`, and `execute_and_evaluate_agent`).
(3) **A working prototype and end-to-end demonstration**[1] on the Base Sepolia testnet, exercising the full flow over external flight- and hotel-search services and reporting end-to-end success rate.

The rest of the paper is organized as follows. Section 2 reviews the protocols UniAgent builds on. Section 3 positions our work against related research, with particular attention to Vaziry et al. Section 4 presents the three-layer architecture, the core of our contribution. Section 5 describes the prototype, and Section 6 evaluates it. Section 7 discusses limitations and future work, and Section 8 concludes.

2 Background

2.1 A2A Protocol

The Agent2Agent (A2A) protocol [10] is an open standard for communication between independent agents. Each agent publishes an *Agent Card*, a JSON document served at a well-known URL (`/.well-known/agent.json`) that describes the agent's name, capabilities, service endpoint, and—in payment-aware deployments—its price. Clients then exchange task messages with the agent over JSON-RPC. A2A deliberately standardizes only the communication format and leaves the question of *how clients discover a suitable agent* to the application. This is the gap that a marketplace must fill.

[1]Source code: https://github.com/toryoto/UniAgent; live demo: https://uni-agent-web.vercel.app/.

2.2 x402 Protocol

x402 [8] is a micropayment protocol that revives the HTTP 402 "Payment Required" status code. When a client requests a paid resource, the server responds with `402` together with machine-readable `PAYMENT-REQUIRED` requirements (amount, asset, recipient, network, and scheme). The client then constructs a payment and retries the request with a `PAYMENT-SIGNATURE` header; the server verifies and settles the payment, typically through a *facilitator* that submits the transaction on chain, and finally returns the resource with a `PAYMENT-RESPONSE` header. Payments are denominated in stablecoins such as USDC and are made gasless for the payer using EIP-3009 *transfer with authorization* [4], which lets a user authorize a transfer with an off-chain signature that a third party submits. We describe how UniAgent applies this pattern in Section 4.3.

2.3 On-Chain Agent Identity (ERC-8004)

ERC-8004 [9] standardizes a registry of agent identities on the Ethereum Virtual Machine. Built on the ERC-721 non-fungible token standard, it assigns each agent a token whose metadata (the Agent Card) is stored off chain and referenced by its content identifier. Because the registry is a public contract, any client can enumerate agents and resolve their metadata without trusting a central catalog, which makes it a natural *single source of truth* for a decentralized marketplace.

2.4 LLM-Based Agent Orchestration

The ReAct pattern [13] interleaves reasoning and acting: the LLM alternates between producing a thought, calling a tool, and observing the result, accumulating context until the task is done. This loop is a good fit for orchestration, because the set of agents, prices, and intermediate results is not known in advance and must be explored at run time. UniAgent implements the orchestration layer as a ReAct agent built with LangChain, exposing three tools—for discovery, specification lookup, and paid execution—so that the model can operate the marketplace autonomously.

3 Related Work

Agent communication protocols. Several protocols now compete to standardize how agents interact. Ehtesham et al. [1] survey this space and compare the Model Context Protocol (MCP), which connects a single agent to tools and data sources, with peer-to-peer protocols such as A2A and the Agent Network Protocol (ANP). UniAgent adopts A2A for inter-agent communication because it is designed for cross-organization delegation, and uses an MCP-style tool interface only *inside* the orchestrator. None of these communication protocols specifies discovery or payment, which is precisely what our marketplace layer adds.

Agent payment systems. x402 [8] attaches stablecoin transfers to ordinary HTTP requests and settles them on a smart-contract chain. We build on x402 because it is open, HTTP-native, and requires no proprietary intermediary, and we extend it with delegated EIP-3009 signing through Privy (Section 4.3).

Decentralized agent identity and closest prior work. Vaziry et al. [11] represent each agent's card as an individual smart contract

and show that A2A, x402, and on-chain identity can interoperate. UniAgent instead consolidates identities into a single standardized ERC-8004 registry [9] rather than per-agent contracts, and—more importantly—builds a full marketplace and orchestration layer on top, as detailed below.

Difference from Vaziry et al. Table 1 summarizes the contrast.

Table 1: UniAgent compared with Vaziry et al. [11].

Aspect	Vaziry et al.	UniAgent
Positioning	Protocol-level feasibility study	Marketplace architecture integrating the protocols
Identity	Per-agent smart contracts	Single ERC-8004 standard registry
Discovery	Four methods proposed, not integrated	Registry plus webhook-synced cache
Payment	x402 over A2A, demonstrated	x402 with delegated signing
Orchestration	None (fixed agent A → B)	LLM ReAct agent discovers, selects, executes
User experience	Protocol-level proof of concept	Chat UI and SSE streaming

Vaziry et al. establish the technical *feasibility* of the protocol combination through one-to-one agent interactions. UniAgent treats those protocols as building blocks and contributes the missing layer above them: an LLM orchestrator that, given a single user task, autonomously discovers agents from a standardized registry, selects among them, pays them over x402, and returns an integrated result—together with the user-facing experience (chat and streaming) that this requires.

LLM multi-agent orchestration. AutoGen [12] shows how multiple LLM-based agents can coordinate through conversation, tool use, and optional human input within an application. UniAgent addresses a different layer: it treats the marketplace itself—open registration, cross-vendor discovery, and protocol-native payment—as the object of orchestration.

Centralized marketplaces. Centralized agent stores such as the OpenAI GPT Store [6] offer registration, discovery, and billing as a smooth, integrated experience, but only within one vendor's platform: agents are listed, ranked, and charged according to platform-specific rules, and there is no portability across providers. Enterprise vendors are moving in the same direction: Oracle AI Agent Marketplace embeds validated, partner-built agent templates directly inside Oracle Fusion Applications and adopts A2A for agent interoperability within that environment [7]. Such systems reinforce the demand for agent marketplaces, but they also show why A2A alone is insufficient: communication may be standardized while registration, discovery, deployment, and billing remain confined to a single vendor's application layer. They also rely on platform-managed billing rather than protocol-native micropayments. This resembles the broader platform pattern of walled gardens, where openness and neutrality are limited by the platform operator [3]. UniAgent aims to reproduce the convenience of such platforms while keeping each function open and vendor-neutral.

4 System Architecture

UniAgent is organized into three layers, shown together in Figure 1: a *marketplace layer* that maintains the catalog of available agents, an *orchestration layer* that turns a user task into a sequence of agent invocations, and a *payment layer* that settles each invocation. The layers are deliberately decoupled—the orchestrator does not need to know how identity is stored, and the payment layer does not need to know why an agent was selected—so that each can evolve independently. We describe each layer in turn and then explain how they combine into an end-to-end flow.

4.1 Marketplace Layer

The marketplace layer answers a single question: which agents exist and what can they do? Its authoritative source is the on-chain ERC-8004 `AgentIdentityRegistry`. Reading directly from the chain on every request, however, would be slow and costly, so UniAgent maintains an off-chain cache and keeps it consistent with the chain through an event-driven pipeline.

Figure 2 shows the developer-facing flow. An agent developer first deploys an A2A agent and uploads its Agent Card to IPFS (Phase A). The developer then registers the agent in the ERC-8004 registry, recording the IPFS content identifier as the token's metadata URI, and optionally stakes a deposit (Phase B). Registration and staking emit on-chain events. A blockchain indexing service (Alchemy) watches for these events and calls a webhook on the web application (Phase C); the webhook verifies the event, fetches the metadata from IPFS, and updates a PostgreSQL cache. Finally, the marketplace UI reads from this cache to list agents quickly and cheaply (Phase D). Because the chain remains the single source of truth and the cache is rebuilt from on-chain events, the cache can be regenerated at any time without loss of authority.

4.2 Orchestration Layer

The orchestration layer is a ReAct agent [13] built with LangChain and driven by a commercial LLM. It receives the user's natural-language task and operates the marketplace through three tools:

- `discover_agents(category, query)` returns a short list of candidate agents for a given need, drawn from the marketplace cache (Section 4.1).
- `fetch_agent_spec(agentId)` retrieves the candidate's Agent Card from its well-known URL and, when available, its OpenAPI specification. This lookup is free of charge and lets the LLM inspect skills, input modes, endpoint details, and x402 pricing before committing to a paid call.
- `execute_and_evaluate_agent(agentId, category, input)` invokes the chosen agent over A2A, settles payment over x402 (Section 4.3), verifies the resulting on-chain transaction, and records a quality attestation through EAS.

Orchestration loop. Given a task such as planning a trip, the ReAct agent first decomposes it into sub-tasks (e.g., a flight search and a hotel search). For each sub-task it calls `discover_agents` to obtain up to three ranked candidates, then typically calls `fetch_agent_spec` on the most promising ones to compare capabilities and prices. After selecting an agent, it calls `execute_and_evaluate_agent`, observes the result, and repeats the loop until the overall task is

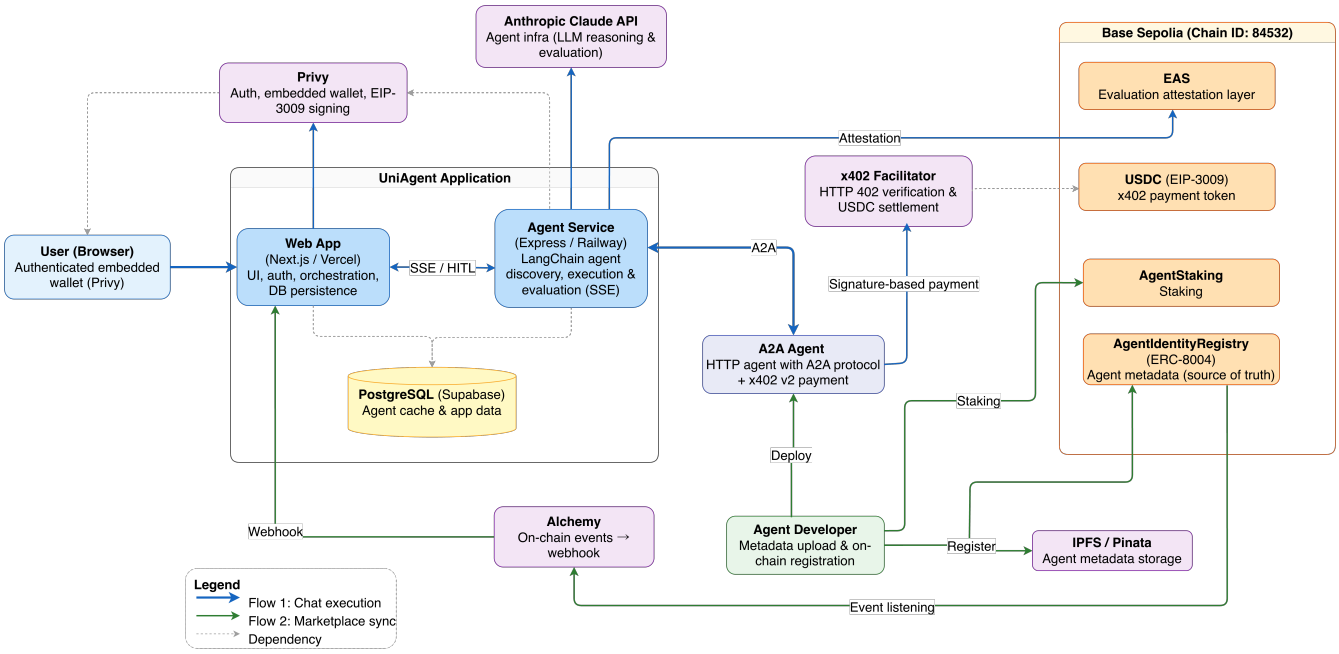


Figure 1: Overview of the UniAgent architecture. A user issues a natural-language task through the web application; an LLM-based agent service discovers candidate agents from an ERC-8004 registry (cached in PostgreSQL via Alchemy webhooks), invokes the selected external A2A agents, and settles payment over x402 using USDC on Base Sepolia.

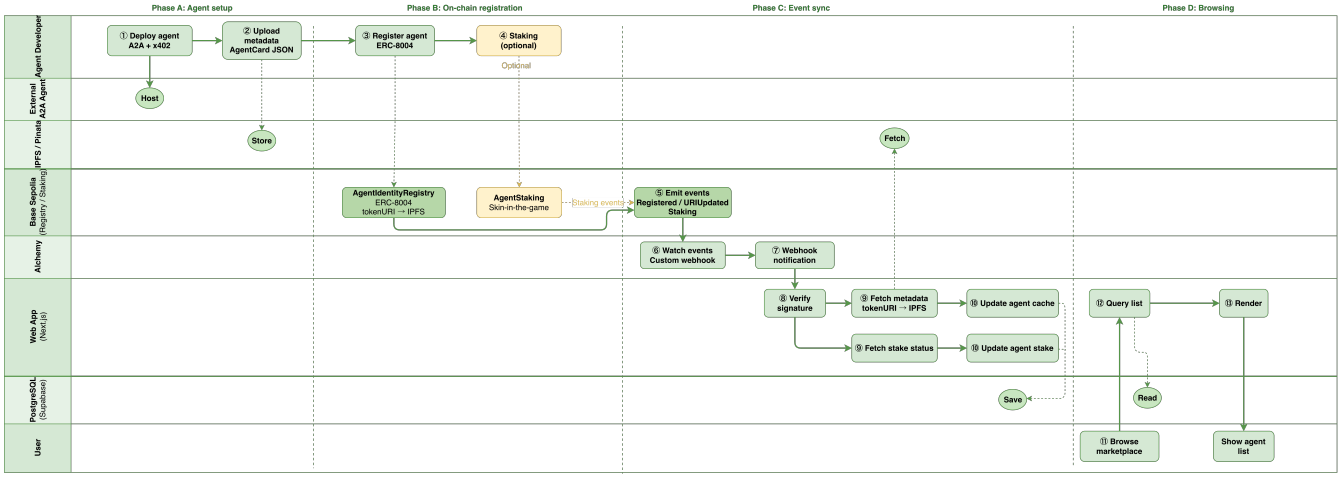


Figure 2: Agent registration and marketplace synchronization. A developer deploys an A2A agent, uploads its Agent Card to IPFS, and registers it in the ERC-8004 registry (optionally staking a deposit). On-chain events are delivered to a webhook through a blockchain indexer, which refreshes the PostgreSQL cache that backs the marketplace UI.

complete, at which point it composes a final answer. Crucially, the model is never given a fixed plan or a hard-coded list of agents—both the decomposition and the choice of providers emerge from the ReAct loop at run time.

Discovery ranking. To keep the candidate list short and useful, `discover_agents` does not return every matching agent. Instead, the marketplace ranks candidates from the cache using a composite score that combines an agent's reliability and quality (estimated from past attestations), the size of its on-chain deposit, and a small bonus for newly registered agents, and returns the top three. The reliability and quality estimates are smoothed toward a global average so that agents with few reviews are neither unfairly promoted nor permanently excluded (mitigating the cold-start problem), and

a small exploration term occasionally surfaces a new agent so that it can accumulate a track record. We describe the ranking qualitatively here, as a way to give the LLM a manageable and balanced set of options; its exact form is an engineering parameter rather than a contribution of this paper.

4.3 Payment Layer

The payment layer settles each paid invocation issued by `execute_and_evaluate_agent` using x402 [8]. Figure 3 illustrates how UniAgent instantiates the x402 verify-and-settle pattern, delegating EIP-3009 signing to the agent service through Privy without giving it custody of user funds. The diagram traces the full exchange among five parties: the agent service (via its internal `execute_agent` handler), a Privy delegated wallet, the external A2A agent, an x402 facilitator, and USDC on the Base Sepolia testnet. The orchestrator first posts an A2A task request to the agent's HTTP endpoint. The agent responds with HTTP 402 and a `PAYMENT-REQUIRED` header that specifies the price, recipient (`payTo`), network, and payment scheme. The agent service then requests an EIP-3009 *transfer with authorization* [4] from Privy; Privy returns a signed payment payload (`from`, `to`, `value`, `nonce`, and signature components) without exposing the user's private key to the server. The service retries the request with the payload in the `PAYMENT-SIGNATURE` header. The external agent verifies the payload through the facilitator's `/verify` endpoint, executes the task, and calls `/settle` so that the facilitator submits `transferWithAuthorization` on chain. Once the transaction is confirmed on Base Sepolia, the agent returns HTTP 200 with the task result and a `PAYMENT-RESPONSE` header.

This sequence follows the standard x402 verify-and-settle pattern; the main UniAgent extension is *delegated signing* through Privy, which makes payments gasless and non-custodial while keeping the user's key under wallet-provider control. After a successful execution, the system records an off-chain quality and reliability attestation using the Ethereum Attestation Service (EAS) [2]; these attestations feed back into the discovery ranking above, closing the loop between payment and reputation. A full evaluation of this feedback loop is left to future work.

4.4 Integration Design

Figure 4 shows how the layers combine into a single end-to-end experience. After the user authenticates with Privy and submits a task, the web application opens a server-sent events (SSE) stream and forwards the task to the orchestrator. The orchestrator runs the ReAct loop—reasoning about the task, discovering agents (marketplace layer), inspecting their Agent Cards through `fetch_agent_spec`, and executing the chosen agent with x402 payment (payment layer). Throughout, intermediate events—reasoning steps, tool calls, payments, and partial results—are streamed back to the UI over SSE, so the user sees the task unfold in real time rather than waiting for a single opaque response.

The value of the design comes from the combination rather than any single protocol. A2A makes agents callable across organizations; ERC-8004 makes them discoverable and identifiable without a central directory; x402 makes them payable with a single signed request; and the orchestration layer ties these together so that a user expresses a goal in natural language and the system carries out the underlying discovery, delegation, and payment automatically.

5 Implementation

We implemented UniAgent as a TypeScript monorepo; the source code is available at https://github.com/toryoto/UniAgent and the live deployment at https://uni-agent-web.vercel.app/. The web application uses Next.js and React, with Privy for social login and delegated wallets. The orchestration service runs separately on LangChain with a commercial LLM, exposing the three tools described in Section 4.2 and streaming results over SSE. The marketplace cache is stored in PostgreSQL and accessed through an ORM, with all database access confined to a dedicated data-access layer. Smart contracts are written in Solidity and developed with Hardhat. Payments use the x402 protocol with USDC on the Base Sepolia testnet.

Deployment. The prototype runs on the Base Sepolia testnet (chain ID 84532). Table 2 lists the main on-chain components: the ERC-8004 `AgentIdentityRegistry` that anchors agent identity, the `AgentStaking` contract that holds agent deposits, and the EIP-3009–capable USDC token in which x402 payments are settled. An EAS schema is additionally registered to record per-interaction quality and reliability attestations.

Table 2: Primary on-chain deployments (Base Sepolia, chain ID 84532).

Contract	Address
AgentIdentityRegistry	`0x864A0C054AA6E9DBcCDB36a44a14A5A7bc81EB92`
AgentStaking	`0xC034e56EDe7FC31579E41095A4e963D499e85d39`
USDC	`0x036CbD53842c5426634e7929541eC2318f3dCF7e`

Demonstration scenario. For the prototype we focus on a travel-planning use case with two agent categories, *flight* and *hotel*, served as external A2A services. A request such as "plan a three-day trip to Paris" is decomposed by the orchestrator into a flight sub-task and a hotel sub-task, each resolved through discovery, specification inspection, and paid execution, and the results are combined into a single itinerary. The evaluated marketplace is intentionally small but heterogeneous. In each category we deploy one A2A agent backed by a real travel-search API and ten dummy A2A agents to exercise discovery, ranking, and provider-quality variation. Thus the evaluation measures protocol-level cross-service integration rather than production-scale provider coverage.

Figures 5a and 5b illustrate the web interface. Figure 5a shows the chat view: the user enters a task in natural language and observes the orchestrator's tool calls— `discover_agents` to rank candidates, `fetch_agent_spec` to compare Agent Cards, and eventually `execute_and_evaluate_agent`—streamed over SSE. Figure 5b shows the marketplace view, where users browse registered agents with category and price filters; each card displays on-chain ERC-8004 metadata and an x402 price badge.

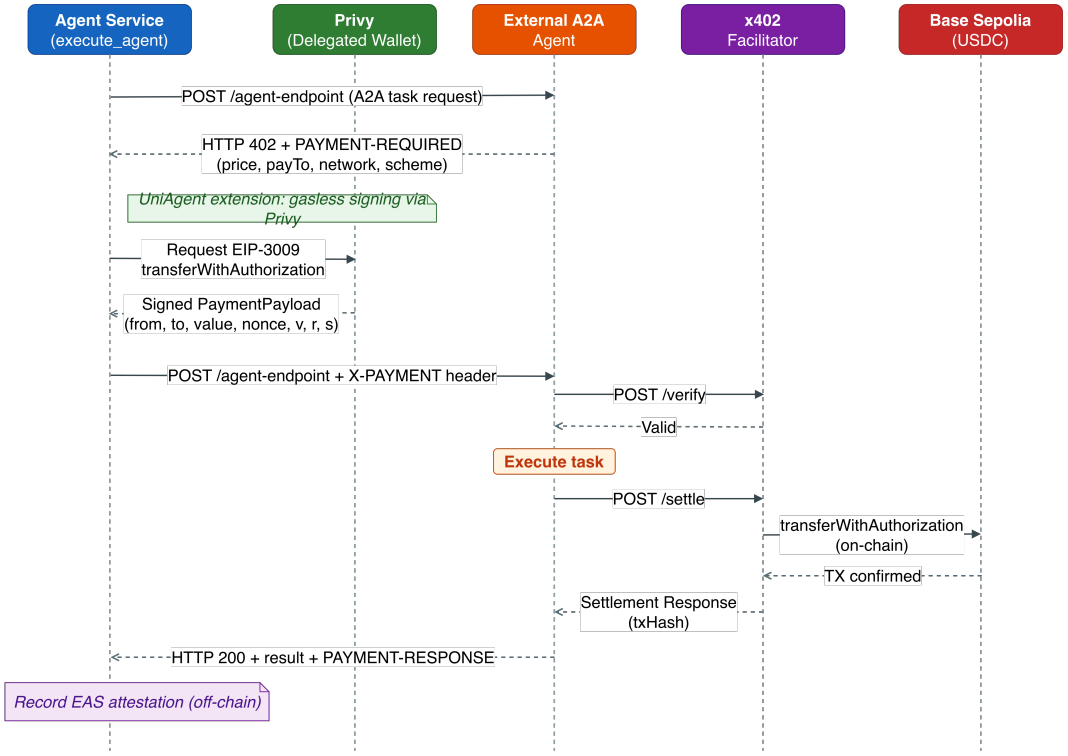


Figure 3: x402 payment sequence in UniAgent. The agent service sends an A2A task request, receives HTTP 402 with `PAYMENT-REQUIRED`, obtains a gasless EIP-3009 signature from a Privy delegated wallet, retries with `PAYMENT-SIGNATURE`, and the external agent verifies and settles the payment through an x402 facilitator on Base Sepolia before returning the result. After settlement, the agent service records an off-chain EAS attestation (not shown as a message arrow).

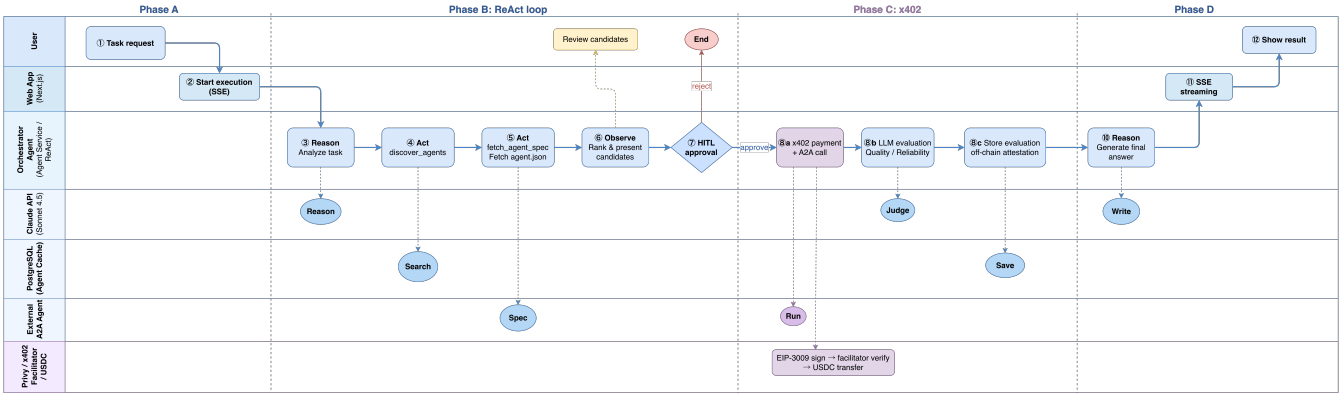
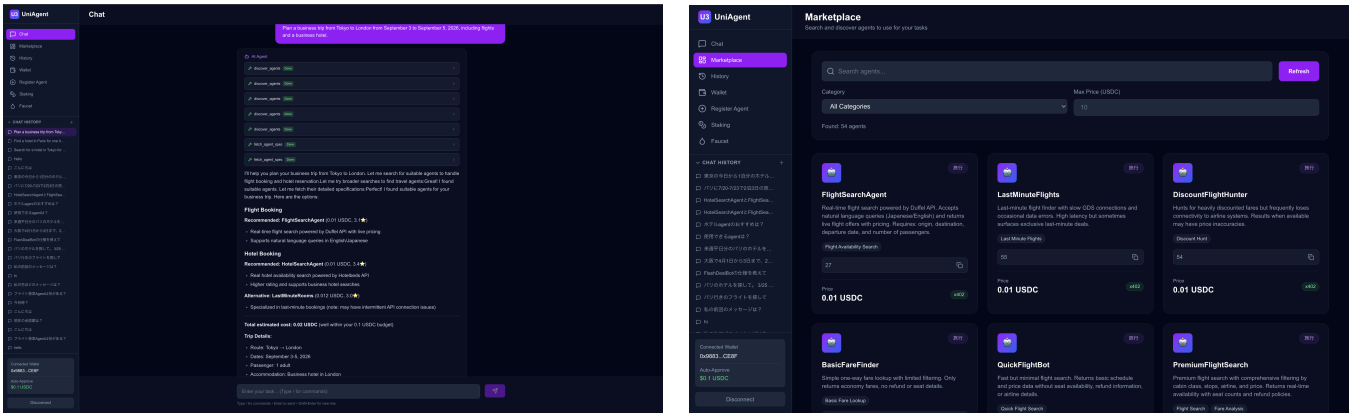


Figure 4: End-to-end execution flow. The orchestrator runs a ReAct loop that discovers, inspects, and executes external A2A agents, settling each call over x402. Progress is streamed to the UI over SSE.

6 Evaluation

Our evaluation asks whether the integrated architecture can carry a realistic user task through discovery, delegation, and payment end to end. The results reported below are from 20 completed runs: ten single-agent hotel tasks and ten multi-agent flight-plus-hotel tasks.

Across all runs we use the same system prompt for the orchestrator. The prompt uses standard prompt-engineering patterns, including role specification, step-wise workflow decomposition, tool-use constraints, safety gates for paid execution, structured output formatting, and few-shot examples. These instructions define the common operating procedure—agent discovery, specification



(a) Chat view: natural-language task input with streamed tool calls and ranked agent recommendations.

(b) Marketplace view: searchable listing of registered agents with ERC-8004 metadata and x402 price badges.

Figure 5: The UniAgent web interface.

lookup, candidate presentation, user approval before paid execution, execution, and final reporting—but do not encode scenario-specific answers, fixed agent choices, expected outputs, or success labels. We judge success from execution logs using the criteria below, rather than relying on the LLM's own self-assessment.

6.1 Experimental Setup

We deploy UniAgent on the Base Sepolia testnet with two categories of externally callable A2A services, flight and hotel. Each category contains one API-backed search agent and ten dummy A2A agents used to populate the marketplace and provide alternative discovery candidates. We define a task set spanning two classes: single-agent tasks and multi-agent tasks that require chaining two agents. For each task we measure the end-to-end success rate.

The task set contains ten prompts per scenario class. We vary destination, travel dates, party size, and travel intent (budget, business, family, or luxury travel) while keeping the required agent categories fixed. The prompts are ordinary natural-language requests; they do not include endpoint URLs, transaction hashes, expected outputs, or success labels. For paid execution, UniAgent first presents candidate agents and requires the user to approve the selected provider before calling `execute_and_evaluate_agent`. In many runs, the user approved the orchestrator-recommended provider by name (for example, `HotelSearchAgent`) rather than with a generic confirmation. This matches the system's human-in-the-loop payment design, but it means the reported results evaluate the approval-mediated end-to-end workflow rather than independently stress-testing provider-selection accuracy. We therefore count a run as successful only if the system completes the full approval-mediated path: discovery, specification lookup, candidate presentation, user approval, paid execution, x402 settlement, and final result reporting.

For each run, we record the selected agents, x402 transaction hashes, and final execution logs, then manually label protocol completion and task success. Provider-output mismatches are noted separately from protocol failures.

6.2 Scenarios and End-to-End Success Rate

We use two scenario classes. S1 tests a single paid agent call using hotel-search tasks. A representative prompt is: "Search for a hotel in Tokyo for 2 adults from July 24 to July 26, 2026." A successful S1 run requires that one hotel agent be discovered, approved, paid, executed, and returned in the final response with hotel results.

S2 tests multi-agent composition. The orchestrator must decompose a travel-planning request into flight and hotel sub-tasks, discover and inspect providers for each role, execute both providers, and integrate their results. A representative prompt is: "Plan a business trip from Tokyo to London from September 3 to September 5, 2026, including flights and a business hotel."

Table 3 reports the observed completion rates for these scenario classes.

Table 3: Observed end-to-end results by scenario.

Scenario	Type	Runs	Protocol	Task
S1	single agent	10	10/10	10/10
S2	multi-agent	10	10/10	8/10

Representative case. We describe S2-02 in detail because it exercises the full multi-provider path. The user asks: "Plan a business trip from Tokyo to London from September 3 to September 5, 2026, including flights and a business hotel." The orchestrator must first decompose the request into flight search and hotel search sub-tasks. It then calls `discover_agents` for each role, retrieves the relevant Agent Cards and OpenAPI specifications with `fetch_agent_spec`, and presents candidate providers with prices before any paid execution. Once the user approves the selected providers, the orchestrator invokes `execute_and_evaluate_agent` for the flight agent and the hotel agent, each with its own x402 payment. A successful run requires two payment transaction hashes, two agent results, and a final response that combines flight and hotel options into

a single business-trip plan. This case therefore covers decomposition, provider selection, human approval, two paid A2A calls, x402 settlement, quality evaluation, and result integration.

The table separates protocol completion from task-level success. Protocol completion means that UniAgent reached paid execution, settled the x402 payment, returned transaction hashes, and produced a final report. Task success additionally requires the external provider results to match the requested city, dates, and role. All S1 and S2 runs completed the protocol path. S1 also achieved 10/10 task success. S2 achieved 8/10 task success: in S2-03, the hotel provider returned mainly Johor Bahru results for a Singapore family trip, and in S2-07 the flight provider returned a Tokyo–London route for a Tokyo–New York request. In both cases, UniAgent still completed discovery, payment, execution, and final reporting, and the orchestrator surfaced the mismatch rather than presenting the output as fully correct. Minor provider-quality issues, such as duplicate flight listings or mixed nearby-city hotel results, were recorded separately from task failures.

6.3 Comparison with Existing Approaches

Table 4 compares UniAgent feature-by-feature with the protocol-level study of Vaziry et al. [11] and with centralized platforms such as the GPT Store [6] and Oracle AI Agent Marketplace [7]. Oracle's adoption of A2A illustrates that agent-interface standards are becoming commercially relevant, but the benefit remains limited if registration, discovery, billing, and deployment stay inside a single vendor's application layer. UniAgent matches the end-to-end convenience of centralized platforms while keeping registration, discovery, and payment open and vendor-neutral, and it adds the orchestration layer that the protocol-level study lacks. The centralized platforms achieve an end-to-end flow only within closed environments, whereas UniAgent does so across separately deployed A2A services.

The evaluation results should be read in this context. UniAgent's strongest measured result is protocol completion: every observed S1 and S2 run reached paid execution and returned verifiable x402 transaction hashes. The task-level failures in S2 were not caused by the marketplace, payment, or A2A integration layers, but by provider output quality. This distinction is important for an open marketplace architecture. The framework can make discovery, payment, execution, and reporting auditable across provider boundaries, while still exposing quality differences among separately deployed services. In contrast, centralized marketplaces can hide those boundaries behind a platform-specific runtime, but they also bind discovery, billing, and orchestration to a single vendor.

Table 4: Feature comparison.

Feature	Vaziry et al.	UniAgent	GPT Store	Oracle Marketplace
Registration	Custom contract	ERC-8004	Platform-specific	Validated templates
Discovery	Proposed, not integrated	Registry + cache	In-platform search	Within Fusion Apps
Payment	x402 PoC	x402 + delegated signing	Platform billing	Platform billing
Orchestration	None	LLM ReAct	Platform-dependent	App-embedded
End-to-end flow	No	Yes	Yes (closed)	Yes (closed)
Vendor-neutral	Yes	Yes	No	No

7 Discussion

The prototype shows that an open, integrated marketplace is feasible: once registration, discovery, communication, and payment are each expressed as open protocols, an LLM orchestrator can stitch them into an autonomous end-to-end flow. We expect the dominant cost to be LLM reasoning and external-agent execution rather than payment, since x402 settlement adds only a single signed request and one on-chain transfer.

UniAgent has several limitations. It currently runs on a testnet, so real economic incentives and adversarial behavior are not yet exercised. The evaluation uses two agent categories (flight and hotel); broader domains and a larger agent population remain to be tested. The discovery ranking and the EAS-based reputation feedback are presented here as design elements and have not been validated at scale. Controlled provider-failure recovery is also left to future evaluation, since the reported runs focus on the completed single-agent and multi-agent execution paths. Finally, end-to-end quality depends on the LLM's selection accuracy: prior benchmarks show that reasoning, decision-making, and instruction following remain key failure modes for LLM agents [5], so a poor choice among candidates can lower task success even when every protocol behaves correctly.

Future work includes large-scale evaluation of the discovery and reputation mechanisms, controlled failure-recovery experiments, hardening the system for mainnet (security and key management for delegated payments), refining the economic model for staking and pricing, and extending orchestration to richer multi-agent workflows.

8 Conclusion

We presented UniAgent, a decentralized agent marketplace that integrates the A2A communication protocol, x402 micropayments, and ERC-8004 on-chain identity under an LLM-based orchestration layer. The main contribution of our architecture is the separation and integration of three responsibilities that are usually handled by closed platforms: a marketplace layer for open agent registration and discovery, an orchestration layer that turns a user request into agent selection and delegation, and a payment layer that settles cross-provider execution through protocol-native micropayments. Whereas prior work demonstrated these protocols in isolation or

as one-to-one interactions, UniAgent shows how they can be composed into a coherent, vendor-neutral framework in which a single natural-language request is autonomously decomposed into agent discovery, delegation, and payment. A working prototype on Base Sepolia demonstrates the full flow over separately deployed flight- and hotel-search services, showing that end-to-end convenience need not depend on a centralized agent platform. We believe this integrated design is a step toward an open "web of agents" in which users delegate tasks across organizational boundaries as easily as they browse the web today.

References

[1] Abul Ehtesham, Aditi Singh, Gaurav Kumar Gupta, and Saket Kumar. 2025. A Survey of Agent Interoperability Protocols: Model Context Protocol (MCP), Agent Communication Protocol (ACP), Agent-to-Agent Protocol (A2A), and Agent Network Protocol (ANP). https://arxiv.org/abs/2505.02279. arXiv:2505.02279. Accessed: 2026-05-31.

[2] Ethereum Attestation Service. 2024. Ethereum Attestation Service (EAS) Documentation. https://docs.attest.org/. Accessed: 2026-05-31.

[3] Rob Frieden. 2016. The Internet of Platforms and Walled Gardens: Implications for Openness and Neutrality. SSRN Working Paper. SSRN:2754583. Accessed: 2026-05-31.

[4] Peter Jihoon Kim, Kevin Britz, and David Knott. 2020. ERC-3009: Transfer With Authorization. https://eips.ethereum.org/EIPS/eip-3009. Ethereum Improvement Proposals, no. 3009, September 2020. Accessed: 2026-05-31.

[5] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. 2024. AgentBench: Evaluating LLMs as Agents. In *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. arXiv:2308.03688.

[6] OpenAI. 2024. Introducing the GPT Store. https://openai.com/index/introducing-the-gpt-store/. Accessed: 2026-05-31.

[7] Oracle. 2025. Oracle Launches Fusion Applications AI Agent Marketplace to Accelerate Enterprise AI Adoption. https://www.oracle.com/news/announcement/ai-world-oracle-launches-fusion-applications-ai-agent-marketplace-to-accelerate-enterprise-ai-adoption-2025-10-15/. Press release. Accessed: 2026-06-10.

[8] Erik Reppel, Ronnie Caspers, Kevin Leffew, Danny Organ, Dan Kim, and Nemil Dalal. 2025. x402: An Open Standard for Internet-Native Payments. https://www.x402.org/x402-whitepaper.pdf. Coinbase Developer Platform whitepaper. Accessed: 2026-05-31.

[9] Marco De Rossi, Davide Crapis, Jordan Ellis, and Erik Reppel. 2025. ERC-8004: Trustless Agents. https://eips.ethereum.org/EIPS/eip-8004. Ethereum Improvement Proposals, no. 8004, August 2025. Accessed: 2026-05-31.

[10] Rao Surapaneni, Miku Jha, Michael Vakoc, and Todd Segal. 2025. Agent2Agent (A2A) Protocol. https://a2a-protocol.org/. Specification, Google. Accessed: 2026-05-31.

[11] Awid Vaziry, Sandro Rodriguez Garzon, and Axel Küpper. 2025. Towards Multi-Agent Economies: Enhancing the A2A Protocol with Ledger-Anchored Identities and x402 Micropayments for AI Agents. https://arxiv.org/abs/2507.19550. arXiv:2507.19550 [cs.MA]. Accessed: 2026-05-31.

[12] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. https://arxiv.org/abs/2308.08155. arXiv:2308.08155. Accessed: 2026-05-31.

[13] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proceedings of the 11th International Conference on Learning Representations (ICLR)*. arXiv:2210.03629.